

ECE 157B/272B - Homework 2

ECE 157/272 TAs

Winter 2024

Due: Friday, February 7, 2025 by 11:59 PM

Introduction

In the last homework, we introduced the Pytorch library's basic recurrent neural network components and the basics of text-based data processing. In this homework, we will be moving into the foundation model domain and introducing the hugging-face (HuggingFace) library [4]. This library contains many text, vision, and audio pre-trained models and datasets.

When we are using large models, we do not want to retrain the models from scratch as doing so requires a large amount of computational power and time. Instead, we will fine-tune a pre-trained model for the task we want to perform.

In this homework, we will follow the hugging-face tutorial (here: <https://huggingface.co/course/chapter7/7?fw=pt>) to build a BERT model [1] for a question answering task. We will see how to leverage the hugging-face tools and pre-trained models to help us process data and fine-tune the BERT model for a question answering task. And we also compare this question answering model with our last quarter built RAG(Retrieval-Augmented Generation). In the end, we will also use BertViz to visualize the attention heads within a BERT model and to examine their internal workings as discussed in class.

hugging-face

hugging-face is a platform where the machine learning community collaborates on models, datasets, and applications. It provides an extensive library of pre-trained large language and vision models. These models, ranging from transformer-based architectures like BERT and GPT to innovative vision models like Vision Transformers and Stable Diffusion, are trained on vast datasets to excel in various tasks such as text classification, sentiment analysis, image recognition, and object detection. With hugging-face's rich selection of pre-trained models, researchers and developers can leverage state-of-the-art capabilities without the need for extensive training from scratch, significantly accelerating the development process and enabling the exploration of cutting-edge techniques in NLP and CV.

Moreover, hugging-face offers a diverse collection of datasets, spanning from common benchmarks like SQuAD to specialized datasets in domains such as healthcare and finance. These datasets serve as invaluable resources for experimentation, benchmarking, and evaluation, facilitating reproducible research and fostering collaboration within the community. Additionally, hugging-face serves as a centralized repository for storing and sharing models, providing a collaborative platform where researchers and practitioners can publish, discover, and explore models contributed by the community. Furthermore, hugging-face offers courses to help users learn how to effectively utilize the pre-trained models for various tasks, enhancing accessibility and usability.

Tutorial Background

In this homework, we want you to follow the hugging-face tutorial on question answering (here). The type of question answering task performed is called extractive question answering, where a question is posed for

a given document and the answer will be a span of text from the document. Throughout the tutorial, you will see examples of the following:

- Loading and examining an existing dataset from the hugging-face **dataset** library
- Processing data by leveraging a pre-trained tokenizer
- Processing data into the format required for the question answering task training
- Fine-tuning a pre-trained model using the hugging-face **Trainer** API (or building a custom training loop, for advanced users)
- Saving your model checkpoints onto the hugging-face hub
- Processing data for validation and leveraging existing metrics from the hugging-face **evaluate** library
- Using the fine-tuned model for question answering

Dataset

SQuAD (Stanford Question Answering Dataset)[2]. This question answering dataset is developed by Stanford. We will be using the version uploaded to a hugging-face dataset (here). A benefit of using the hugging-face dataset library over the raw dataset is that the dataset is structured into a **DatasetDict** and preliminary processing has been done for us. You can find information about the dataset, such as column descriptions and example samples, on the hugging-face dataset page above.

SQuAD has the following columns:

- **id** (str): The identifier of a sample
- **title** (str): The title of a document
- **context** (str): The text of a document
- **question** (str): The question to ask about a document
- **answers** (dict[list]): A dictionary containing two lists. One list is the answer text span, and the other list is the answer character starting index

Tasks

1. Hugging-face QA Tasks

We will be following the hugging-face tutorial to build a question answering model, which we will use to answer some questions about the tutorial itself. The steps are listed below:

1. Carefully read through the tutorial and try to understand what steps are performed and the purpose of each step. You don't need to completely understand the code; instead, focus on the conceptual ideas.
2. Train a question answering model as instructed in the tutorial.
3. For the report, answer the questions about the tutorial and make a note of
 - The answers, in your own words.
 - The document context which is the paragraph you got your answer from.
 - A screenshot of your fine-tuned model's output when you pass the context and question.
 - A comment on the model's answer. Did the model answer correctly? Example responses: *"The model's answer is correct, short and concise."*, *"The model's answer is partially correct; it missed the portion about [something]."*

hugging-face Tutorial Questions

1. What dataset is used in this tutorial?
2. What is tricky about processing the answer field of the dataset?
3. How are the labels for the answer formatted?
4. What type of tokenizer is needed?
5. How do we deal with long context?
6. What is the stride parameter?
7. What will be the label if the answer got truncated in the splitting process?
8. What is the purpose of `overflow_to_sample_mapping`?
9. What is the post-process trying to do?
10. What do you need to push the trained model to the Hub?

2. Review of Retrieval-Augmented Generation and Compare

In the last quarter, we covered building the RAG (Retrieval-Augmented Generation). For this assignment, you have the option to either utilize the Streamlit app developed last quarter or the RAG code provided. Fit in the Hugging Face tutorial PDF and compare the performance with the BERT model.

Building a RAG (skip this part if you already built the RAG model last quarter)

With encoders and generative models, you will create an application that can read PDFs. The application then uses LLMs to answer questions based on the PDFs, otherwise known as retrieval-augmented generation (RAG).

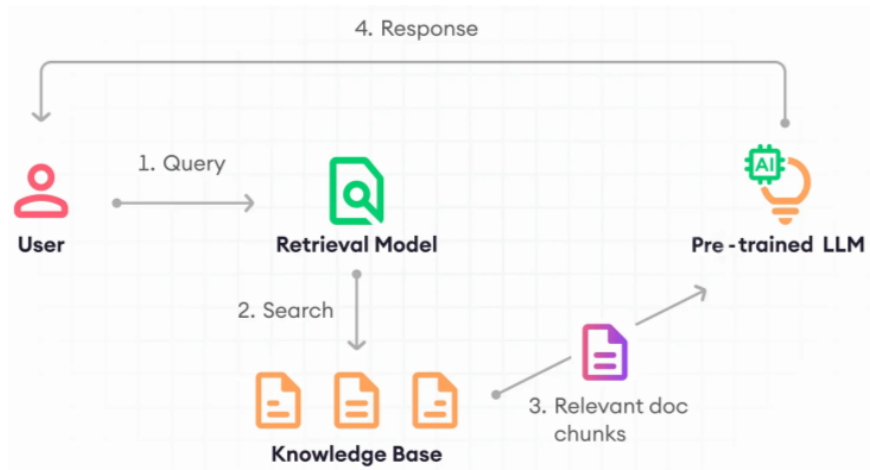


Figure 1: A diagram for retrieval-augmented-generation (RAG)

By using Ollama, it can care of everything downloading, quantization, formatting, installation. It will also automatically handle GPU usage, if you have it.

1. Download and install from Ollama website.
2. Check that the Ollama python library was properly installed from our requirements.txt with **import ollama**. Here is the link of introduction to ollama here.

3. Open a second terminal in Vscode (or on your system directly Windows, Linux, Mac) and run **ollama serve**. This will create a ollama server at port 11434. Also, **ollama pull [model_name]** if needed. (For this assignment, the **llama3.2:latest** model is sufficient, and it is smaller in size.)
4. Complete the TODOs in **rag_qa.py** before running the python file.

3. BERT Attention Head Visualization Tasks

In this part of the homework, we will use BertViz (here) to visualize the attention head of the BERT model. *It does not have to be the BERT model you trained in the previous step. You may work on this with a different BERT model while your model is training.*

In [3], the authors show that BERT rediscovers the traditional NLP (natural language processing) pipeline which consists of POS tagging, parsing, NER, semantic roles, and coreference. We will perform a similar probing experiment to validate the noun modifier experiment mentioned in the class. We will be using the following two sentences as the input for individual probing experiments:

- The complicated language in the huge new law has muddled the fight.
- The 45-year-old former General Electric Co. executive figures it will be easier this time.

Your task is to find **one** head that, for both sentences, is able to show similar results as the one shown in Lecture 3 slide 71 (Reproduced in Fig. 2).

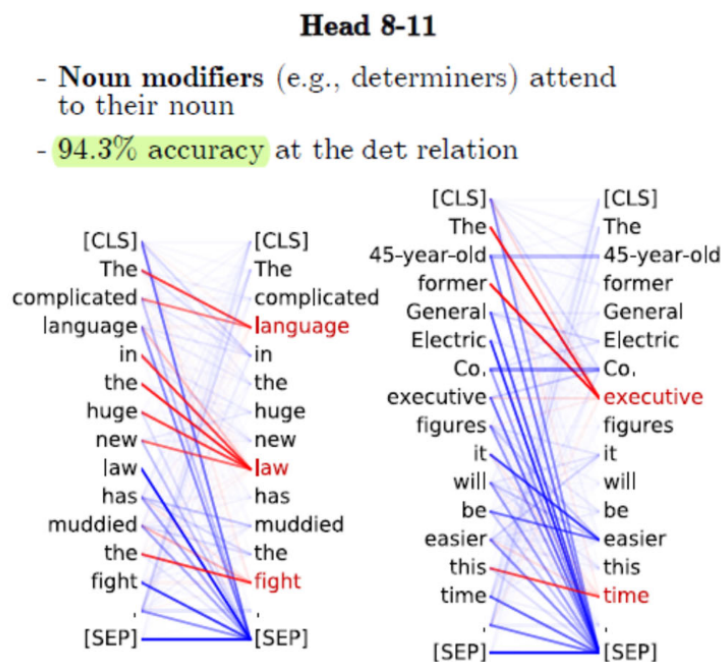


Figure 2: Noun Modifier Output from Lecture 3 Slide 71

The experiment code is provided. You just need to change **sentence_a** to one of the two sentences and iterate through all the layers and all the heads to find the head that performs the noun modifier operation. For the report, Show the following:

- The BERT model that is used.
- The attention layer and head that performs the noun modifier operation.
- Screenshots of the attention for this head (one for each sentence).

What to turn in

- For the report, use the given slide template to make *slides* that show the results/submissions requested in the task sections. Export and submit the slides as a PDF document to GradeScope.

References

- [1] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2018. DOI: 10.48550/ARXIV.1810.04805. URL: <https://arxiv.org/abs/1810.04805>.
- [2] Pranav Rajpurkar et al. “SQuAD: 100,000+ Questions for Machine Comprehension of Text”. In: *arXiv e-prints*, arXiv:1606.05250 (2016). arXiv: 1606.05250.
- [3] Ian Tenney, Dipanjan Das, and Ellie Pavlick. “BERT Rediscovered the Classical NLP Pipeline”. In: *CoRR* abs/1905.05950 (2019). arXiv: 1905.05950. URL: <http://arxiv.org/abs/1905.05950>.
- [4] Thomas Wolf et al. “Transformers: State-of-the-Art Natural Language Processing”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Online: Association for Computational Linguistics, Oct. 2020, pp. 38–45. URL: <https://www.aclweb.org/anthology/2020.emnlp-demos.6>.